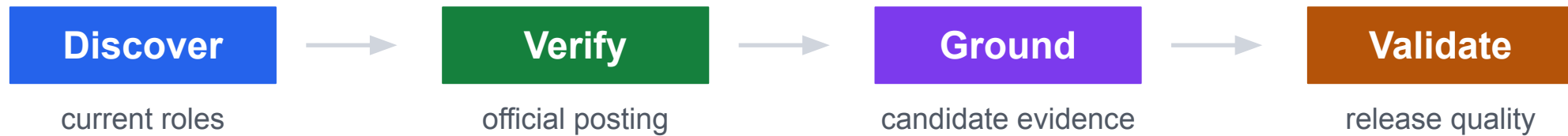


Career Application Agent

A stateful, evidence-grounded workflow for finding roles, verifying openings, and generating reviewable application packages.



Project output: resume · cover letter · notes · provenance · application history

AI drafting needs controls

The agent separates role status, candidate evidence, and quality gates.

Stale openings

Search results and copied job descriptions are not proof that a role is still open.

Unsupported claims

A job description lists desired skills, not facts about the candidate.

Unreviewed output

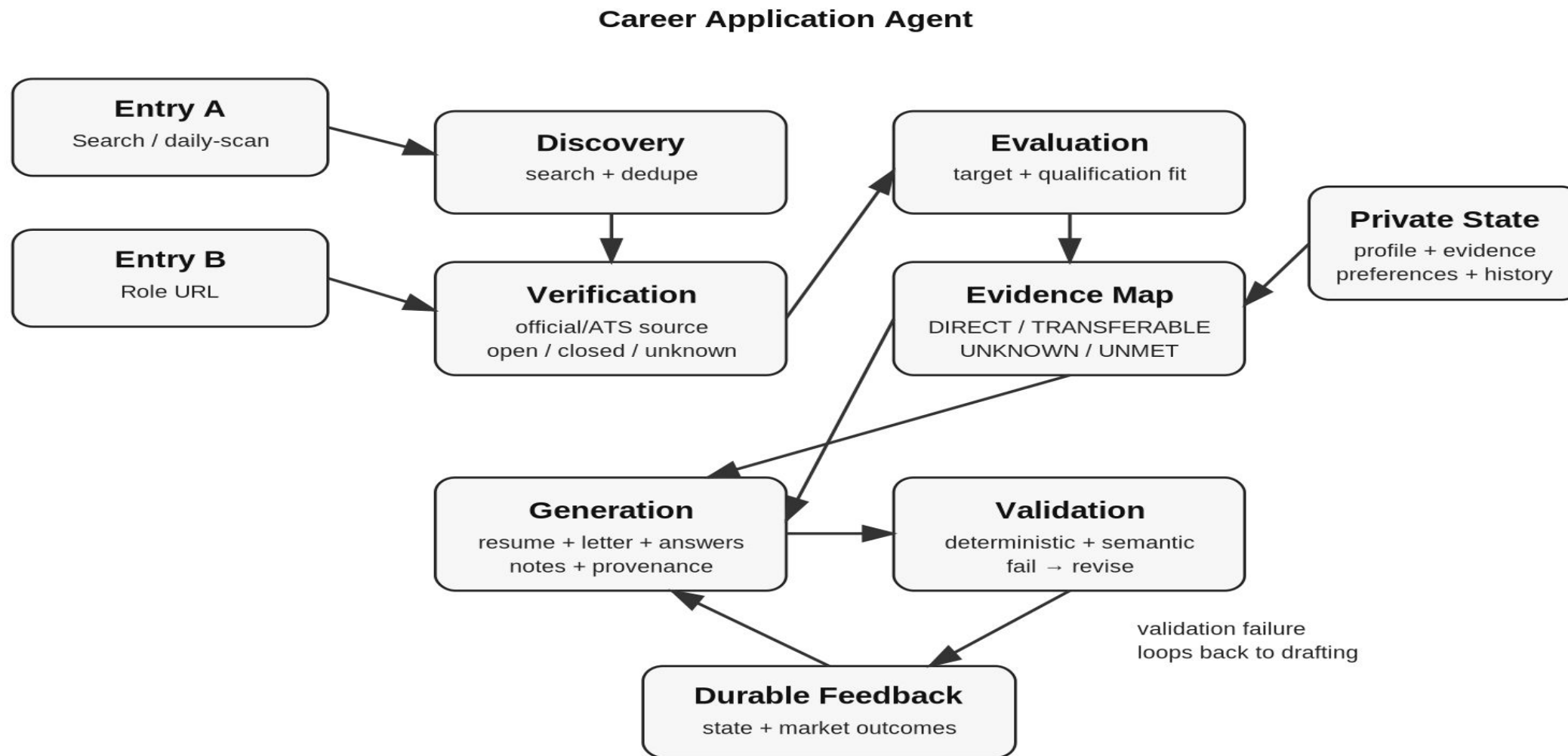
A fluent resume can still contain broken links, chronology drift, or inflated scope.

Design principle

Use AI for interpretation and drafting; keep facts, opening status, and release quality behind evidence and deterministic checks.

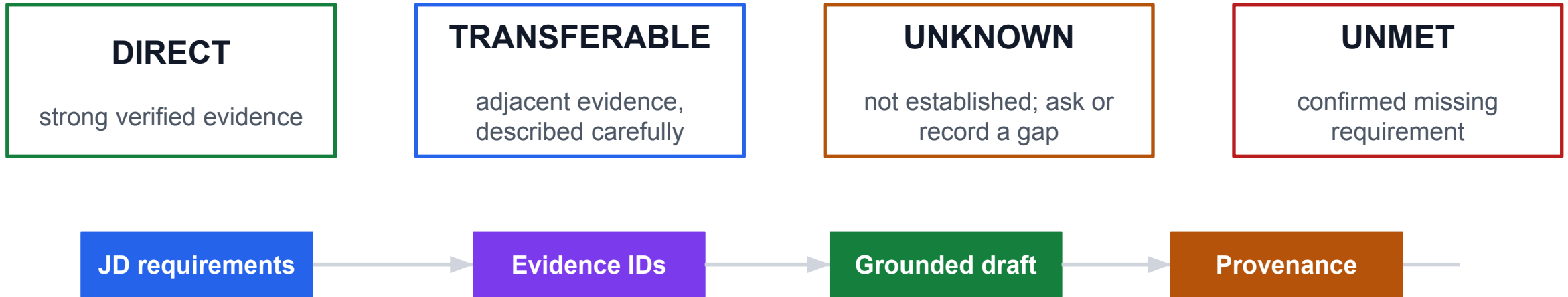
End-to-end workflow

Two entry paths feed the same evidence-grounded application pipeline.



Evidence grounding

The job description is role evidence, not candidate evidence.



Controls prevent named-tool leakage, invented metrics, date drift, unsupported ownership claims, and cross-document inconsistencies.

What to review in the repository

The repo is designed to be skimmed visually, then inspected through samples and tests.

README

quick overview + setup

docs/architecture.svg

workflow diagram

samples/

fictional end-to-end run

skills/

reusable agent procedures

scripts/ + tests/

release gates and regression tests

Fast takeaway

This is not a single resume prompt.

It is a stateful agent workflow with explicit evidence boundaries, opening verification, validation, and durable application history.

Design choices

The agent is designed around failure modes, not just prompt convenience.

1. Ground claims in evidence

The job description can influence selection and wording, but it cannot create candidate facts. Claims must trace back to verified or user-confirmed evidence.

2. Separate reasoning from invariants

LLM skills handle interpretation, fit analysis, and drafting. Code enforces hard boundaries such as evidence IDs, link integrity, privacy, and state rules.

3. Fail to unknown

If a job opening, tool claim, metric, date, or qualification cannot be verified, the system records a gap instead of inventing certainty.

Use AI for judgment and drafting; use checks for facts, safety, and release quality.

How to use and verify it

The repository is meant to be inspected through example data, workflow traces, and release gates.

Use path

- Copy example profile/evidence files into local ignored files.
- Start with a search goal or a specific role URL.
- Verify official opening status before drafting.
- Map JD requirements to DIRECT / TRANSFERABLE / UNKNOWN / UNMET.
- Generate reviewable materials, then validate before release.

Mechanical validation

Schemas, required files, private-file guardrails, hyperlinks, and state-history merge behavior.

Grounding validation

Evidence IDs must exist; unknown or unmet requirements cannot become candidate claims.

Human/semantic review

A reviewer checks relevance, scope, tone, and whether the output is ready to submit.

Release gate: `python scripts/release_gate.py`